

Progetto di Reti Logiche

Prof. Fornaciari, Prof. Palermo e Prof. Salice
Anno Accademico 2024 - 2025

SPECIFICA per lo svolgimento del progetto

Revisione del 17 dicembre 2024

Descrizione generale

La specifica della "Prova Finale (Progetto di Reti Logiche)" per l'Anno Accademico 2023/2024 chiede di implementare un modulo HW (descritto in VHDL) che si interfacci con una memoria e che rispetti le indicazioni riportate nella seguente specifica.

Il sistema legge, un byte alla volta, un messaggio in memoria costituito da una sequenza di K parole W, il cui valore è compreso tra -128 e +127 rappresentato in complemento a 2, applica un filtro differenziale e salva la sequenza di K parole R in memoria.

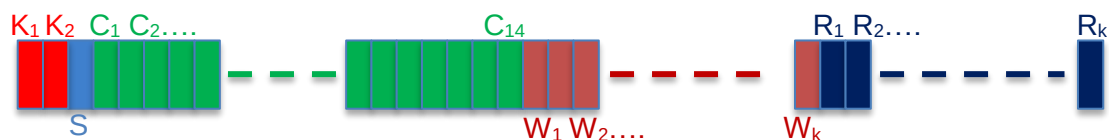
Tutti i dati da utilizzare, di partenza, sono memorizzati sequenzialmente a partire da un indirizzo specificato (ADD). I dati iniziali sono i seguenti:

- **17 (2+1+14) byte** che indicano la lunghezza della sequenza, l'ordine del filtro differenziale e i coefficienti. In particolare:
 - **K1 e K2**, due byte, che indicano la lunghezza della sequenza K dei dati W; K1 è il byte più significativo; **la lunghezza minima è di 7 byte.**
 - **S**, un byte, che indica quale filtro selezionare tra quello di ordine 3 e quello di ordine 5; Il valore consente di selezionare le aree di memoria nelle quali sono caricati i coefficienti. Se il bit meno significativo di S è pari a 0 il filtro da utilizzare è quello di ordine 3, se il bit meno significativo di S è pari a 1 il filtro da utilizzare è quello di ordine 5.
 - **Da C1 a C14**, quattordici byte, che memorizzano i coefficienti dei due filtri. In particolare:
 - Ordine 3: $c=[0, -1, 8, 0, -8, 1, 0]$ normalizzazione $n=12$
 - Ordine 5: $c=[1, -9, 45, 0, -45, 9, -1]$ normalizzazione $n=60$
- **K byte** da elaborare che contengono valori da -128 a +127.

Il filtro ha la seguente funzione $f'(i) = (1/n) * \sum C_j * f[j+i]$ con j che va da -l e +l e con l che vale 2 per il filtro di ordine 3 e 3 per il filtro di ordine 5; n è il valore di normalizzazione. Si osservi che il risultato del filtraggio prima della normalizzazione ($\sum C_j * f[j+i]$) richiede un numero di bit adeguato per la sua rappresentazione.

Per esempio, per il filtro di ordine 3 la $f'(x) \approx (1/12)*(-f[i-2]+8*f[i-1]-8*f[i+1]+f[i+2])$.

La sequenza K del risultato R viene riscritta a partire dall'indirizzo $ADD+17+K$ cioè alla fine dei valori da elaborare.



I valori ad inizio e fine sequenza sono da considerare pari a 0. Per esempio, considerando un filtro di ordine 3, i valori iniziali $ADD+17-2$, $ADD+17-1$ non esistono e il filtro utilizza degli 0. Allo stesso modo, a fine sequenza $ADD+17+K$ e $ADD+17+K+1$ non esistono e il filtro utilizza degli 0.

Per esempio, si consideri la sequenza **+101 -86 -12 +41 +87 -123 -127 +87 +108**. Utilizzando un filtro di ordine 3, il primo valore ottenuto è **$0 \cdot -1 + 0 \cdot 8 + 101 \cdot 0 - 86 \cdot -8 - 12 \cdot 1$ cioè -40**.

Ogni valore ottenuto deve essere normalizzato dividendolo per il coefficiente n ; per esempio, 60 quando si utilizza il filtro di ordine 5. La divisione deve essere approssimata nel seguente modo:

- $1/12$ è approssimato con $1/16 + 1/64 + 1/256 + 1/1024$
- $1/60$ è approssimato con $1/64 + 1/1024$

Importante: si osservi che ogni *shift a destra* equivale a una divisione per due e produce un troncamento verso -1 per numeri negativi e verso 0 per i numeri positivi. Quindi, nel caso di numeri negativi, l'errore si amplifica perché ogni termine viene troncato indipendentemente, come avviene nel caso degli shift multipli, e può fornire un contributo di -1. Per esempio, se il numero (in decimale) è -1, ogni shift destro (per esempio, $1/16$, $1/64$, $1/256$, $1/1024$) restituisce -1 e la somma è -4. Per ridurre questo errore, si aggiunge +1 al risultato del singolo shift quando il valore da shiftare è negativo. Riassumendo, se $val \gg m$ è maggiore di zero si aggiunge 0, se $val \gg m$ è minore di zero si aggiunge 1.

Per esempio, $-1 \cdot 1/12$ è circa uguale a $-1 \cdot (1/16 + 1/64 + 1/256 + 1/1024) = (-1 \gg 4 + 1) + (-1 \gg 6 + 1) + (-1 \gg 8 + 1) + (-1 \gg 10 + 1) = 0$.

Per l'esempio precedente, invece, $-40 \cdot (1/12)$ può essere approssimato con $(-40 \gg 4 + 1) + (-40 \gg 6 + 1) + (-40 \gg 8 + 1) + (-40 \gg 10 + 1) + (-40 \gg 12 + 1)$ cioè, in *Complemento a 2* 1111 1101 1000 si ottiene $(1111\ 1111\ 1101 + 0000\ 0000\ 0001) + (1111\ 1111\ 1111\ 1111 + 0000\ 0000\ 0001) + (1111\ 1111\ 1111\ 1111 + 0000\ 0000\ 0001) + (1111\ 1111\ 1111\ 1111 + 0000\ 0000\ 0001) + (1111\ 1111\ 1111\ 1111 + 0000\ 0000\ 0001) = 1111\ 1111\ 1110$. Il risultato è poi riportato su 8 bit cioè 1111 1110 (-2 decimale al posto del valore corretto -3 ma meglio -5, ottenuto se non applicassimo la correzione).

ESEMPIO(valori in decimale)

- Sequenza di partenza

32 -24 -35 0 46 -54 -39 -22 -53 -47 12 11 11 45 -30 -14 -35 -25 -19 -35 -41 -61 -24 -62

- Dati filtrati con filtro di ordine 3

157 536 -178 -678 428 658 -355 119 251 -487 -400 100 -314 303 426 ...

- Sequenza finale

11 43 -13 -54 33 53 -28 8 18 -38 -31 7 -24 23 33

Funzionamento

Il modulo da implementare ha **3 ingressi primari**, uno ad **1 bit (START)**, uno a **16 bit (ADD)** e una **uscita primaria da 1 bit (DONE)**. Inoltre, il modulo ha un segnale di clock **CLK**, unico per tutto il sistema, e un segnale di reset **RESET** anch'esso unico per tutto il sistema. Tutti i

segnali sono sincroni e devono essere interpretati sul fronte di salita del clock. L'unica eccezione è RESET che, invece, è asincrono.

All'istante iniziale, quello relativo al **reset** del sistema, l'uscita DONE deve essere 0. Quando RESET torna a zero, il modulo partirà nella elaborazione quando un segnale START in ingresso verrà portato a 1. Il segnale di START rimarrà alto fino a che il segnale di DONE non verrà portato alto; al termine della computazione (e una volta scritto il risultato in memoria), il modulo da progettare deve alzare (portare a 1) il segnale DONE che notifica la fine dell'elaborazione. Il segnale DONE deve rimanere alto fino a che il segnale di START non è riportato a 0. Un nuovo segnale START non può essere dato fin tanto che DONE non è stato riportato a zero.

Il modulo deve essere progettato considerando che prima del primo START=1 (e prima di richiedere il corretto funzionamento del modulo) verrà sempre dato il RESET (RESET=1). Prima del primo reset del sistema, il funzionamento del modulo da implementare è non specificato. Una seconda (o successiva) elaborazione con START=1 non dovrà invece attendere il reset del modulo. Ogni qual volta viene dato il segnale di RESET (RESET=1), il modulo viene re-inizializzato.

Quando il segnale di START viene posto ad 1 (e per tutto il periodo in cui esso rimane alto) sugli ingressi ADD vengono posti il primo indirizzo da elaborare. Il modulo prima di alzare il segnale di DONE deve aggiornare la sequenza ed i relativi valori di credibilità al valore opportuno seguendo la descrizione generale del modulo.

Nota: Se il primo dato della sequenza è pari a zero, il suo valore rimane tale e il valore di credibilità deve essere posto a 0 (zero). Lo stesso succede fino al raggiungimento del primo dato della sequenza con valore diverso da zero.

Interfaccia del Componente

Il componente da descrivere deve avere la seguente interfaccia.

```
entity project_reti_logiche is
  port (
    i_clk  : in std_logic;
    i_rst  : in std_logic;
    i_start : in std_logic;
    i_add  : in std_logic_vector(15 downto 0);

    o_done : out std_logic;

    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_data : out std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
  );
end project_reti_logiche;
```

In particolare:

- il nome del modulo **deve essere** `project_reti_logiche` e deve essere presente **una sola architettura** per ogni entità; la violazione di queste indicazioni comporta l'impossibilità di eseguire il Test Bench e una conseguente valutazione di zero;
- `i_clk` è il segnale di CLOCK in ingresso generato dal Test Bench;
- `i_rst` è il segnale di RESET che inizializza la macchina pronta per ricevere il primo segnale di START;
- `i_start` è il segnale di START generato dal Test Bench;
- `i_add` è il segnale (vettore) ADD generato dal Test Bench che rappresenta l'indirizzo dal quale parte la sequenza da elaborare;
- `o_done` è il segnale DONE di uscita che comunica la fine dell'elaborazione;
- `o_mem_addr` è il segnale (vettore) di uscita che manda l'indirizzo alla memoria;
- `i_mem_data` è il segnale (vettore) che arriva dalla memoria e contiene il dato in seguito ad una richiesta di lettura;
- `o_mem_data` è il segnale (vettore) che va verso la memoria e contiene il dato che verrà successivamente scritto;
- `o_mem_en` è il segnale di ENABLE da dover mandare alla memoria per poter comunicare (sia in lettura che in scrittura);
- `o_mem_we` è il segnale di WRITE ENABLE da dover mandare alla memoria (=1) per poter scriverci. Per leggere da memoria esso deve essere 0.

APPENDICE: Descrizione Memoria

NOTA: La memoria è già istanziata all'interno del Test Bench e non va sintetizzata

La memoria e il suo protocollo può essere estratto dalla seguente descrizione VHDL che fa parte del test bench e che è derivata dalla User guide di VIVADO disponibile al seguente link:

https://www.xilinx.com/support/documentation/sw_manuals/xilinx2017_3/ug901-vivado-synthesis.pdf

```
-- Single-Port Block RAM Write-First Mode (recommended template)
--
-- File: rams_02.vhd
--
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity rams_sp_wf is
port(
    clk  : in  std_logic;
    we   : in  std_logic;
    en   : in  std_logic;
    addr : in  std_logic_vector(15 downto 0);
    di   : in  std_logic_vector(7 downto 0);
    do   : out std_logic_vector(7 downto 0)
);
end rams_sp_wf;

architecture syn of rams_sp_wf is
type ram_type is array (65535 downto 0) of std_logic_vector(7 downto 0);
signal RAM : ram_type;
begin
    process(clk)
    begin
        if clk'event and clk = '1' then
            if en = '1' then
                if we = '1' then
                    RAM(conv_integer(addr)) <= di;
                    do <= di after 2 ns;
                else
                    do <= RAM(conv_integer(addr)) after 2 ns;
                end if;
            end if;
        end if;
    end process;
end syn;
```

ESEMPI

Gli esempi qui di seguito (**valori in esadecimale**) hanno lo scopo di esemplificare la relazione tra la dei valori della sequenza di partenza (indicata come “prima”), cioè durante il periodo nel quale START assume valore 1, il valore ottenuto dal filtraggio (indicata come “ $\sum C_i * f[j+i]$ ”) e la configurazione finale (indicata come “dopo”) nel solo momento utile, cioè durante il periodo nel quale DONE assume valore 1. Il valore R è scritto dopo la sequenza delle parole W. Inizialmente R è 0. K è il numero di parole W della sequenza mentre ADD è l'indirizzo nel quale si trova la prima parola W. I valori sono scritti sia in esadecimale che decimale.

ESEMPIO 1 Filtro ordine 3

K1	00							
K2	06							
S	0E							
ADD	00CB							
Prima		$\sum C_i * f[j+i]$				Dopo		
ADDw	W	Wdec	exa	dec		ADDR	R	Rdec
00CB	6B	107	D2	-814		00D1	BF	-65
00CC	61	97	19	1049		00D2	56	86
00CD	DA	-38	46	1606		00D3	84	132
00CE	91	-111	A0	-864		00D4	BB	-69
00CF	31	49	6C	-404		00D5	E0	-32
00D0	B9	-71	B6	1462		00D6	77	119

ESEMPIO 2 Filtro ordine 5

K1	00							
K2	0F							
S	0F							
ADD	01A8							
Prima		$\sum C_i * f[j+i]$				Dopo		
ADDw	W	Wdec	exa	dec		ADDR	R	Rdec
01A8	F0	-16	BE	-834		01B7	F3	-13
01A9	73	115	11	-751		01B8	F5	-11
01AA	56	86	AF	431		01B9	06	6
01AB	41	65	57	343		01BA	05	5
01AC	0F	15	B6	1206		01BB	13	19
01AD	92	-110	B0	944		01BC	0E	14
01AE	94	-108	33	-973		01BD	F1	-15
01AF	19	25	5C	-1700		01BE	E5	-27
01B0	7A	122	A8	168		01BF	02	2
01B1	1E	30	CB	203		01C0	03	3
01B2	64	100	5E	-418		01C1	FA	-6
01B3	34	52	34	1844		01C2	1D	29
01B4	88	-120	C7	-569		01C3	F8	-8
01B5	72	114	20	-1248		01C4	EC	-20
01B6	1B	27	75	1141		01C5	12	18

ESEMPIO 3 Filtro ordine 3

K1	00							
K2	20							
S	80							
ADD	00E3							
Prima		$\Sigma C_i \cdot f(j+i)$				Dopo		
ADDW	W	Wdec	exa	dec		ADDR	R	Rdec
00E3	F3	-13	74	-908		0103	B7	-73
00E4	7F	127	78	-904		0104	B7	-73
00E5	6C	108	42	578		0105	2F	47
00E6	40	64	66	358		0106	1C	28
00E7	3D	61	97	-361		0107	E4	-28
00E8	6D	109	FE	-514		0108	D6	-42
00E9	6B	107	F5	1525		0109	7C	124
00EA	AE	-82	1A	282		010A	16	22
00EB	3A	58	CF	-817		010B	BE	-66
00EC	FF	-1	02	1026		010C	55	85
00ED	C2	-62	D0	-48		010D	FE	-2
00EE	F0	-16	1C	284		010E	16	22
00EF	92	-110	13	787		010F	40	64
00F0	9B	-101	F2	-1294		0110	96	-106
00F1	2D	45	92	-110		0111	F9	-7
00F2	BA	-70	51	337		0112	1B	27
00F3	1C	28	88	-1400		0113	8E	-114
00F4	64	100	47	327		0114	1A	26
00F5	05	5	F1	241		0115	12	18
00F6	49	73	D9	-551		0116	D4	-44
00F7	35	53	19	1049		0117	56	86
00F8	BD	-67	42	834		0118	44	68
00F9	BE	-66	F5	-267		0119	EB	-21
00FA	D3	-45	A5	-91		011A	FA	-6
00FB	DA	-38	34	-716		011B	C7	-57
00FC	42	66	3D	-1219		011C	9C	-100
00FD	6A	106	F2	1522		011D	7C	124
00FE	90	-112	55	341		011E	1B	27
00FF	3C	60	4F	-1201		011F	9E	-98
0100	27	39	5A	-422		0120	DF	-33
0101	71	113	CF	1231		0121	64	100
0102	92	-110	61	97		0122	07	7

ESEMPIO 4 Filtro ordine 5

K1	00							
K2	2F							
S	01							
ADD	0369							
Prima		$\sum C_i \cdot f(j+i)$				Dopo		
ADDW	W	Wdec	exa	dec	ADDR	R	Rdec	
0369	CC	-52	F1	-15	0398	00	0	
036A	F5	-11	69	361	0399	05	5	
036B	99	-103	0D	269	039A	04	4	
036C	D1	-47	5E	-162	039B	FE	-2	
036D	B9	-71	24	-988	039C	F1	-15	
036E	53	83	00	0	039D	00	0	
036F	CD	-51	FB	-261	039E	FC	-4	
0370	71	113	2F	303	039F	04	4	
0371	A4	-92	41	577	03A0	09	9	
0372	3A	58	94	-1644	03A1	E6	-26	
0373	56	86	7E	1406	03A2	16	22	
0374	95	-107	24	804	03A3	0C	12	
0375	FA	-6	B3	-1869	03A4	E2	-30	
0376	7E	126	4E	-630	03A5	F6	-10	
0377	51	81	9F	1695	03A6	1B	27	
0378	9B	-101	8B	1419	03A7	17	23	
0379	81	-127	37	55	03A8	00	0	
037A	89	-119	2F	-977	03A9	F1	-15	
037B	F8	-8	9E	158	03AA	02	2	
037C	82	-126	2D	301	03AB	04	4	
037D	E7	-25	B4	-1356	03AC	EA	-22	
037E	2E	46	A0	-96	03AD	FF	-1	
037F	0C	12	E4	-284	03AE	FC	-4	
0380	4A	74	3E	830	03AF	0C	12	
0381	AB	-85	8F	-113	03B0	FF	-1	
0382	64	100	BA	-1606	03B1	E6	-26	
0383	6B	107	EA	746	03B2	0B	11	
0384	04	4	F2	1522	03B3	18	24	
0385	95	-107	03	771	03B4	0C	12	
0386	A6	-90	9B	-1893	03B5	E2	-30	
0387	7E	126	E2	-286	03B6	FC	-4	
0388	E7	-25	9F	159	03B7	02	2	
0389	7F	127	55	-939	03B8	F2	-14	
038A	4D	77	36	1078	03B9	11	17	
038B	03	3	C3	-61	03BA	00	0	
038C	3D	61	A5	421	03BB	06	6	
038D	C2	-62	60	608	03BC	09	9	
038E	EA	-22	C0	-64	03BD	00	0	
038F	CB	-53	AF	-593	03BE	F7	-9	
0390	45	69	04	-1020	03BF	F1	-15	
0391	49	73	32	818	03C0	0C	12	
0392	DE	-34	D9	985	03C1	0F	15	
0393	C5	-59	EF	-273	03C2	FC	-4	
0394	FE	-2	28	-984	03C3	F1	-15	
0395	38	56	88	904	03C4	0E	14	
0396	9E	-98	38	-200	03C5	FD	-3	
0397	4D	77	1D	-483	03C6	F9	-7	

ESEMPIO 5 Filtro ordine 3

K1	00							
K2	5A							
S	A0							
ADD		0139						
Prima		$\sum C_i \cdot f[j+i]$				Dopo		
ADDw	W	Wdec	ewa	dec		ADDR	R	Rdec
0139	40	64		9F	-609	0193	CF	-49
013A	3D	61		C7	1479	0194	79	121
013B	87	-121		09	521	0195	2A	42
013C	FF	-1		49	-1719	0196	74	-140
013D	59	89		2C	44	0197	02	2
013E	16	22		37	-201	0198	F1	-15
013F	6B	107		37	567	0199	2D	45
0140	C6	-58		A0	672	019A	36	54
0141	10	16		4F	-177	019B	F3	-13
0142	DE	-34		DE	-802	019C	BF	-65
0143	7A	122		1D	-227	019D	EF	-17
0144	F4	-12		76	1398	019E	72	114
0145	DD	-35		F3	-1037	019F	AB	-85
0146	6C	108		1B	-741	01A0	C5	-59
0147	2D	45		7B	1915	01A1	9C	156
0148	8F	-113		C8	-568	01A2	D3	-45
0149	70	112		67	-1433	01A3	8B	-117
014A	4C	76		E2	-30	01A4	FF	-1
014B	7C	124		FC	764	01A5	3C	60
014C	D1	-47		A3	1699	01A6	8B	139
014D	94	-108		48	72	01A7	05	5
014E	AF	-81		38	-200	01A8	F1	-15
014F	B4	-76		99	-615	01A9	CF	-49
0150	09	9		4B	-437	01AA	DE	-34
0151	FD	-3		5F	-417	01AB	DF	-33
0152	42	66		9B	155	01AC	0B	11
0153	DB	-37		57	1367	01AD	70	112
0154	94	-108		09	-247	01AE	EE	-18
0155	E4	-28		9E	158	01AF	0B	11
0156	93	-109		11	-1007	01B0	B0	-80
0157	71	113		74	-908	01B1	B7	-73
0158	0D	13		B0	688	01B2	37	55
0159	28	40		93	147	01B3	0B	11
015A	FB	-5		CA	-566	01B4	D3	-45
015B	74	116		E0	-544	01B5	D5	-43
015C	37	55		79	1145	01B6	5D	93
015D	E8	-24		CA	202	01B7	0F	15
015E	14	20		F2	-526	01B8	D6	-42
015F	26	38		80	-128	01B9	F8	-8
0160	19	25		14	1300	01BA	6B	107
0161	90	-112		57	-681	01BB	CA	-54
0162	78	120		67	-1945	01BC	61	-159
0163	75	117		C9	1737	01BD	8E	142
0164	A8	-88		43	1091	01BE	5A	90
0165	D9	-39		2C	-468	01BF	DB	-37
0166	DB	-37		5D	-675	01C0	CA	-54
0167	39	57		47	-190	01C1	F3	-13

0166	DD	-37		3D	-673		01C0	CA	-34
0167	39	57		42	-190		01C1	F3	-13
0168	05	5		9F	-353		01C2	E4	-28
0169	6B	107		D7	-41		01C3	FE	-2
016A	0A	10		6E	366		01C4	1C	28
016B	38	56		80	384		01C5	1F	31
016C	DB	-37		74	-396		01C6	E1	-31
016D	73	115		CF	-1073		01C7	A8	-88
016E	56	86		46	1094		01C8	5A	90
016F	DF	-33		2F	1583		01C9	81	129
0170	81	-127		EE	-274		01CA	EA	-22
0171	FA	-6		BF	-1089		01CB	A6	-90
0172	1C	28		0E	-754		01CC	C4	-60
0173	76	118		90	-624		01CD	CF	-49
0174	6F	111		89	649		01CE	34	52
0175	22	34		63	611		01CF	31	49
0176	05	5		A2	1186		01D0	61	97
0177	89	-119		F9	-519		01D1	D6	-42
0178	49	73		D3	-1325		01D2	94	-108
0179	3B	59		85	-123		01D3	F8	-8
017A	68	104		BE	446		01D4	22	34
017B	06	6		04	4		01D5	00	0
017C	5F	95		C3	-61		01D6	FD	-3
017D	F7	-9		3A	1338		01D7	6D	109
017E	B3	-77		5C	92		01D8	06	6
017F	E0	-32		38	-712		01D9	C7	-57
0180	03	3		1C	540		01DA	2B	43
0181	AF	-81		30	-464		01DB	DC	-36
0182	47	71		90	-1136		01DC	A4	-92
0183	30	48		E1	1505		01DD	7B	123
0184	9B	-101		80	-128		01DE	F8	-8
0185	30	48		B0	-336		01DF	E6	-26
0186	C7	-57		88	-120		01E0	F8	-8
0187	40	64		06	262		01E1	15	21
0188	A3	-93		0C	268		01E2	15	21
0189	16	22		03	259		01E3	15	21
018A	83	-125		61	-159		01E4	F5	-11
018B	43	67		A6	-1882		01E5	66	-154
018C	6C	108		21	545		01E6	2C	44
018D	04	4		AC	1452		01E7	76	118
018E	AC	-84		38	56		01E8	03	3
018F	EF	-17		18	-744		01E9	C5	-59
0190	FC	-4		EF	751		01EA	3B	59
0191	9C	-100		78	-136		01EB	F6	-10
0192	03	3		E5	-27		01EC	FF	-1

ESEMPIO 6 Filtro ordine 5

K1	00							
K2	80							
S	01							
ADD	00C3							
Prima		$\sum C_i \cdot f[j+i]$				Dopo		
ADDW	W	Wdec	exa	dec	ADDR	R	Rdec	
00C3	9B	-101	BB	443	0143	06	6	
00C4	CA	-54	A1	-863	0144	F3	-13	
00C5	0B	11	CA	-566	0145	F8	-8	
00C6	21	33	27	-217	0146	FD	-3	
00C7	1D	29	7E	1150	0147	12	18	
00C8	81	-127	7A	1146	0148	12	18	
00C9	89	-119	F7	-1033	0149	EF	-17	
00CA	FB	-5	82	-638	014A	F7	-9	
00CB	E4	-28	69	361	014B	05	5	
00CC	DB	-37	19	-231	014C	FD	-3	
00CD	F2	-14	52	850	014D	0D	13	
00CE	84	-124	DA	-1062	014E	EF	-17	
00CF	7E	126	73	-1165	014F	ED	-19	
00D0	15	21	45	1349	0150	16	22	
00D1	ED	-19	79	-331	0151	FA	-6	
00D2	41	65	EE	-786	0152	F4	-12	
00D3	57	87	40	-192	0153	FE	-2	
00D4	53	83	8E	1166	0154	13	19	
00D5	BD	-67	E6	486	0155	07	7	
00D6	FF	-1	6A	106	0156	01	1	
00D7	9D	-99	83	643	0157	0A	10	
00D8	BD	-67	AD	-1107	0158	EE	-18	
00D9	30	48	E9	-1047	0159	EF	-17	
00DA	44	68	11	1041	015A	11	17	
00DB	BE	-66	FC	-4	015B	00	0	
00DC	3E	62	D2	-558	015C	F8	-8	
00DD	FC	-4	AC	428	015D	06	6	
00DE	06	6	A0	672	015E	0A	10	
00DF	AA	-86	00	-512	015F	F9	-7	
00E0	4E	78	0F	-1265	0160	EC	-20	
00E1	3C	60	F7	1527	0161	18	24	
00E2	A5	-91	42	-190	0162	FE	-2	
00E3	59	89	06	-1786	0163	E4	-28	
00E4	78	120	AE	1198	0164	13	19	
00E5	DA	-38	D0	208	0165	03	3	
00E6	5B	91	17	-1001	0166	F1	-15	
00E7	41	65	C5	1221	0167	14	20	
00E8	C7	-57	63	355	0168	05	5	
00E9	FF	-1	33	51	0169	00	0	
00EA	AE	-82	EA	746	016A	0B	11	
00EB	AC	-84	E6	-794	016B	F4	-12	
00EC	19	25	09	-1015	016C	F1	-15	
00ED	3D	61	96	-106	016D	FF	-1	
00EE	3F	63	CD	-563	016E	F8	-8	
00EF	72	114	9A	1434	016F	17	23	
00F0	8E	-114	69	105	0170	01	1	
00F1	4F	79	62	-158	0171	FF	-2	

00F0	8C	-114	63	105	0170	01	1
00F1	4F	79	62	-158	0171	FE	-2
00F2	90	-112	8B	907	0172	0E	14
00F3	E4	-28	DC	-548	0173	F8	-8
00F4	C1	-63	90	400	0174	06	6
00F5	B3	-77	EB	235	0175	03	3
00F6	98	-104	41	321	0176	05	5
00F7	87	-121	09	9	0177	00	0
00F8	A2	-94	00	-1024	0178	F1	-15
00F9	0C	12	31	-207	0179	FD	-3
00FA	C0	-64	1D	797	017A	0C	12
00FB	A8	-88	07	263	017B	04	4
00FC	9F	-97	2B	-725	017C	F5	-11
00FD	0B	11	7B	-645	017D	F6	-10
00FE	03	3	2E	-466	017E	F9	-7
00FF	43	67	59	857	017F	0D	13
0100	8D	-115	B1	1201	0180	13	19
0101	B4	-76	AD	-1363	0181	EA	-22
0102	3C	60	67	-1433	0182	E9	-23
0103	68	104	24	1316	0183	15	21
0104	94	-108	F1	1521	0184	18	24
0105	98	-104	4D	-179	0185	FE	-2
0106	AD	-83	E3	-1821	0186	E3	-29
0107	7D	125	D7	215	0187	03	3
0108	9F	-97	7D	1149	0188	12	18
0109	FF	-1	2C	-1492	0189	E8	-24
010A	3A	58	E4	996	018A	0F	15
010B	81	-127	C8	1224	018B	14	20
010C	93	-109	9D	-99	018C	FF	-1
010D	8F	-113	7E	-1410	018D	E9	-23
010E	47	71	38	56	018E	00	0
010F	9F	-97	D2	-46	018F	00	0
0110	4B	75	43	67	0190	01	1
0111	81	-127	8F	1423	0191	17	23
0112	9A	-102	87	-377	0192	FB	-5
0113	A6	-90	97	-617	0193	F7	-9
0114	FA	-6	ED	-787	0194	F4	-12
0115	18	24	97	-105	0195	FF	-1
0116	17	23	3F	-193	0196	FD	-3
0117	25	37	87	903	0197	0E	14
0118	A1	-95	CF	463	0198	07	7
0119	EF	-17	44	-1212	0199	ED	-19
011A	36	54	D8	-296	019A	FC	-4
011B	11	17	50	1360	019B	16	22
011C	89	-119	B1	433	019C	06	6
011D	D7	-41	9C	-1124	019D	EE	-18
011E	17	23	2E	-466	019E	F9	-7
011F	1D	29	A9	425	019F	06	6
0120	E7	-25	68	104	01A0	01	1
0121	00	0	EA	490	01A1	07	7
0122	97	-105	76	886	01A2	0D	13
0123	87	-121	F0	-16	01A3	00	0
0124	95	-107	87	-633	01A4	F7	-9
0125	E0	-32	A8	-600	01A5	F7	-9
0126	E6	-26	E3	483	01A6	07	7
0127	B7	-73	42	-446	01A7	FA	-6
0128	30	48	86	-1402	01A8	FA	-22

0121	D1	713		42	-7440		01A1	EA	-70
0128	30	48		86	-1402		01A8	EA	-22
0129	72	114		BE	-66		01A9	FF	-1
012A	44	68		58	600		01AA	09	9
012B	15	21		BF	1215		01AB	13	19
012C	A0	-96		73	-141		01AC	FE	-2
012D	11	17		3B	59		01AD	00	0
012E	97	-105		BD	189		01AE	02	2
012F	08	8		DF	-1057		01AF	EF	-17
0130	15	21		50	336		01B0	05	5
0131	E0	-32		C5	965		01B1	0F	15
0132	A7	-89		79	-903		01B2	F2	-14
0133	5D	93		0F	-1521		01B3	E8	-24
0134	76	118		47	71		01B4	01	1
0135	67	103		7C	380		01B5	05	5
0136	3E	62		56	598		01B6	09	9
0137	19	25		EA	-278		01B7	FC	-4
0138	5C	92		6C	-404		01B8	FA	-6
0139	41	65		30	816		01B9	0C	12
013A	EA	-22		DC	988		01BA	0F	15
013B	B9	-71		95	-107		01BB	FF	-1
013C	F8	-8		54	-1196		01BC	ED	-19
013D	46	70		26	806		01BD	0C	12
013E	A6	-90		1E	30		01BE	00	0
013F	4F	79		4F	-1457		01BF	E9	-23
0140	5E	94		77	119		01C0	01	1
0141	55	85		91	145		01C1	02	2
0142	4D	77		8D	-115		01C2	FF	-1